

Start coding or [generate](#) with AI.

LAB

Step-1: create the data set

Generate Realistic House Price Data

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
np.random.seed(42)
n = 200
locations = ['Urban', 'Suburban', 'Rural']
df = pd.DataFrame({
    'area_sqft': np.random.randint(500, 4000, n),
    'num_rooms': np.random.randint(1, 7, n),
    'location': np.random.choice(locations, n),
    'age_years': np.random.randint(0, 40, n),
})
# Location multipliers
loc_map = {'Urban': 1.5, 'Suburban': 1.0, 'Rural': 0.7}#expensiv medium cheap
noise = np.random.normal(0, 20000, n)
df['price'] = (
    df['area_sqft'] * 150 +
    df['num_rooms'] * 25000 -
    df['age_years'] * 2000 +
    df['location'].map(loc_map) * 50000 +
    noise
).astype(int)
print(df.head(8).to_string(index=False))
print(f"\nDataset shape : {df.shape}")
print(df.describe()[['area_sqft', 'num_rooms', 'age_years', 'price']].round(0))
```

```
area_sqft  num_rooms  location  age_years  price
3674         4      Rural        20  646067
1360         4      Urban        35  300479
1794         6      Urban         9  501609
1630         6      Rural        36  355668
1595         3      Urban         8  391991
3592         2      Rural        23  566413
2138         4  Suburban        34  410974
2669         1      Urban        34  459191
```

Dataset shape : (200, 5)

```
area_sqft  num_rooms  age_years  price
count      200.0      200.0      200.0      200.0
mean      2334.0         3.0        20.0    451548.0
std         992.0         2.0        11.0    154871.0
min         521.0         1.0         0.0     86373.0
25%        1519.0         2.0        10.0    326188.0
50%        2372.0         3.0        20.0    458282.0
75%        3230.0         5.0        30.0    575910.0
max        3999.0         6.0        39.0    781261.0
```

Step 2 — Encode & Prepare Features

```
# Encode location: Urban=2, Suburban=1, Rural=0
#Creates a tool that converts text → numbers
le = LabelEncoder()
#transform Converts them into numbers automatically
df['location_enc'] = le.fit_transform(df['location'])
# Select features and target
features = ['area_sqft', 'num_rooms', 'age_years', 'location_enc']
X = df[features]
y = df['price']
# Peek at encoded data
print(df[['location', 'location_enc']].drop_duplicates().sort_values('location_enc'))
print(f"\nFeature matrix shape : {X.shape}")
print(f"Target vector shape : {y.shape}")
```

```
location  location_enc
0      Rural           0
6  Suburban           1
1      Urban           2
```

```
Feature matrix shape : (200, 4)
Target vector shape : (200,)
```

Step 3 — Train / Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Training samples : {len(X_train)} ({len(X_train)/len(X)*100:.0f}%)")
print(f"Testing samples : {len(X_test)} ({len(X_test)/len(X)*100:.0f}%)")
print(f"Feature names : {features}")
```

```
Training samples : 160 (80%)
Testing samples : 40 (20%)
Feature names : ['area_sqft', 'num_rooms', 'age_years', 'location_enc']
```

Step 4 — Train the Model

```
model = LinearRegression()
model.fit(X_train, y_train)
print("Learned Coefficients:")
print("-" * 45)
for feat, coef in zip(features, model.coef_):
    direction = "↑" if coef > 0 else "↓"
    print(f" {feat:15s}: Rs {coef:>10,.0f} {direction}")
print(f" {'intercept':15s}: Rs {model.intercept_:>10,.0f}")
print("-" * 45)
# Interpretation:
# area_sqft → every extra sq ft adds ~Rs 150 to price
# num_rooms → every extra room adds ~Rs 25,000
# age_years → every year older reduces by ~Rs 2,000
```

Learned Coefficients:

```
-----
area_sqft      : Rs      151 ↑
num_rooms      : Rs    23,169 ↑
age_years      : Rs    -2,262 ↓
location_enc   : Rs    18,114 ↑
intercept      : Rs    46,789
-----
```

Step 5 — Evaluate

```
y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("=" * 40)
print(" MODEL EVALUATION REPORT")
print("=" * 40)
print(f"R² Score : {r2:.4f} {'✓ Excellent' if r2>0.9 else '✓ Good' if r2>0.8 else '■ Needs work'}")
print(f"MAE : Rs {mae:>10,.0f}")
print(f"RMSE : Rs {rmse:>10,.0f}")
print("=" * 40)
# Show first 8 predictions vs actuals
print("\nSample Predictions:")
print(f"{'Actual':>10} | {'Predicted':>10} | {'Error':>10}")
print("-" * 36)
for a, p in zip(y_test[:8], y_pred[:8]):
    print(f"{'a':>10,} | {'p':>10,.0f} | {'p-a:>+10,.0f}")
```

```
=====
MODEL EVALUATION REPORT
=====
R² Score : 0.9872 ✓ Excellent
MAE : Rs      13,439
RMSE : Rs     16,959
=====
```

Sample Predictions:

Actual	Predicted	Error
489,899	478,320	-11,579
601,329	572,877	-28,452
623,788	650,489	+26,701
291,458	271,086	-20,372
390,040	344,431	-45,609
123,884	134,545	+10,661
484,442	443,295	-41,147
562,823	556,350	-6,473

Step 6 — Predict on New Houses

```
# New houses to price
new_houses = pd.DataFrame({
    'area_sqft': [1200, 2500, 800, 3500],
    'num_rooms': [3, 5, 1, 6 ],
    'age_years': [5, 10, 25, 2 ],
    'location_enc':[2, 1, 0, 2 ], # 2=Urban,1=Suburban,0=Rural
    'location_lbl':['Urban','Suburban','Rural','Urban'],
})
predictions = model.predict(new_houses[features])
print("\n HOUSE PRICE PREDICTIONS")
print("=" * 60)
for i, (_, row) in enumerate(new_houses.iterrows()):
    print(f" House {i+1}: {row.area_sqft} sqft | "
          f"{row.num_rooms} rooms | "
          f"{row.age_years}yr old | "
          f"{row.location_lbl}")
    print(f" Estimated Price → Rs {predictions[i]:>10,.0f}")
    print()
```

```
HOUSE PRICE PREDICTIONS
=====
House 1: 1200 sqft | 3 rooms | 5yr old | Urban
Estimated Price → Rs      321,914

House 2: 2500 sqft | 5 rooms | 10yr old | Suburban
Estimated Price → Rs      534,588

House 3: 800 sqft | 1 rooms | 25yr old | Rural
Estimated Price → Rs      133,872

House 4: 3500 sqft | 6 rooms | 2yr old | Urban
Estimated Price → Rs      744,552
```